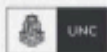


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Telemetría



mundosE



FCEFYN

mundosE

Manual para el alumno: Telemetría.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Telemetría, observabilidad y monitoreo.....	3
2. Métricas: qué son y cómo se recolectan.....	4
3. Logs y su rol dentro de la telemetría.....	6
4. Prometheus e instrumentación de aplicaciones.....	8
5. Grafana, dashboards y análisis de tendencias.....	9
6. Uso de los datos para identificar cuellos de botella y optimizar el rendimiento.....	11
Bibliografía.....	13

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Telemetría

Objetivo del encuentro:

Comprender los conceptos fundamentales de telemetría, métricas y registros (*logs*) y su rol en la operación de aplicaciones y sistemas. Al finalizar el encuentro, reconocerás la importancia de la telemetría dentro de la observabilidad, diferenciarla del monitoreo tradicional y entender cómo utilizar herramientas como Prometheus y Grafana para recolectar, almacenar y visualizar métricas, de manera de detectar problemas de rendimiento, identificar cuellos de botella e interpretar tendencias en el tiempo para tomar decisiones informadas.

Contenido a desarrollar:

- ✓ Definición de telemetría y su rol en la operación de sistemas.
- ✓ Comparación entre telemetría, observabilidad y monitoreo.
- ✓ Métricas: qué son, tipos principales y cómo se recolectan.
- ✓ Registros (*logs*): importancia y relación con las métricas.
- ✓ Prometheus: concepto, arquitectura y recolección de métricas.
- ✓ Instrumentación básica de aplicaciones para exponer métricas.
- ✓ Grafana: visualización, *dashboards* y análisis de tendencias.
- ✓ Uso de los datos recolectados para identificar cuellos de botella.
- ✓ Interpretación de métricas y tendencias a lo largo del tiempo.

1. Telemetría, observabilidad y monitoreo

La palabra *telemetría* significa, literalmente, “medir a distancia”. En sistemas informáticos se utiliza para referirse al conjunto de datos que un sistema emite acerca de su propio comportamiento: métricas numéricas, registros (*logs*) y trazas que describen qué está ocurriendo en tiempo real o qué ocurrió en el pasado reciente. Estos datos permiten ver “desde afuera” el estado de una aplicación o infraestructura, sin necesidad de conectarse manualmente a cada servidor para ejecutar comandos de diagnóstico.

La telemetría es la base de un concepto más amplio, la **observabilidad**. Diversas fuentes la definen como la capacidad de entender el estado interno de un sistema a partir de los datos que produce externamente. Es decir, un sistema observable es aquel que, a partir de sus métricas, *logs* y trazas, permite responder preguntas nuevas sobre su comportamiento sin necesidad de modificar el código cada vez que surge una duda.

Por ejemplo, ante una queja de usuarios que reportan lentitud en determinado horario, un sistema observable debería permitir investigar, con la telemetría ya disponible, si el problema se concentra en una región, en un tipo de operación particular, en un servicio específico o en una dependencia externa (como una base de datos o una API de terceros).

El **monitoreo** es un conjunto de prácticas y herramientas que se apoyan en la telemetría para vigilar la salud del sistema y disparar alertas cuando se detectan condiciones anómalas. Históricamente, se lo asoció con chequeos simples: verificar si un servidor responde al ping, controlar que CPU y memoria no superen ciertos umbrales, o confirmar que un puerto HTTP está disponible. En entornos modernos, el monitoreo se extiende hacia métricas más ricas: latencia, tasa de errores, tráfico por servicio, indicadores de negocio, entre otros.

Es importante entender la relación entre estos tres conceptos:

- La **telemetría** es la materia prima: datos que describen el comportamiento del sistema.
- El **monitoreo** es un uso específico de esa materia prima para vigilar condiciones predefinidas, construir paneles y generar alertas.
- La **observabilidad** es la propiedad de poder formular y responder preguntas complejas sobre el sistema usando la telemetría disponible.

En la práctica, muchas organizaciones arrancan con una forma básica de monitoreo centrada en infraestructura (CPU, memoria, espacio en disco). Esto es necesario, pero claramente insuficiente. En sistemas distribuidos, basados en microservicios, con dependencias externas y tráfico variable, es imprescindible complementar ese enfoque con telemetría que incluya métricas de rendimiento

detalladas y también métricas de negocio (por ejemplo, cantidad de pedidos completados por minuto o porcentaje de pagos fallidos).

Desde la perspectiva de DevOps, un punto central es que el diseño de la telemetría debe formar parte del diseño del sistema. No se trata de agregar “logs al final”, sino de planificar qué señales se necesitan para entender si la solución cumple sus objetivos funcionales y no funcionales. Esa telemetría será luego la base para construir monitoreo y observabilidad.

En síntesis, la telemetría responde a la pregunta “¿qué está pasando en el sistema?”, el monitoreo mira esa información de forma continua para avisar cuándo algo se sale de lo normal, y la observabilidad permite explicar por qué ocurre lo que ocurre, apoyándose en datos y no sólo en intuiciones.

Material para profundizar

- OpenTelemetry. “Observability primer”.
<https://opentelemetry.io/docs/concepts/observability-primer/>
- OpenTelemetry. “OpenTelemetry Concepts”.
<https://opentelemetry.io/docs/concepts/>
- Google SRE Book – Capítulo “Monitoring Distributed Systems”.
<https://sre.google/sre-book/monitoring-distributed-systems/>

2. Métricas: qué son y cómo se recolectan

Las **métricas** son uno de los pilares de la telemetría y, en general, la primera forma de dato que se utiliza al construir soluciones de observabilidad. Una métrica es un valor numérico que describe alguna característica del sistema en un momento determinado. Cuando se registran valores sucesivos a lo largo del tiempo, se forma una **serie temporal**, que permite estudiar tendencias, patrones y anomalías.

En herramientas como Prometheus, una métrica suele estar compuesta por:

- Un **nombre** que la identifica (por ejemplo, `http_requests_total`).
- Un conjunto de **etiquetas** (labels) clave/valor, como `method="GET"`, `status="200"`, `service="checkout"`.
- Un **valor numérico**, por ejemplo, la cantidad acumulada de requests.
- Un **timestamp**, que indica cuándo se registró ese valor.

Cada combinación específica de nombre y etiquetas define una serie temporal distinta. Esto quiere decir que

`http_requests_total{service="checkout", status="200"}` y

`http_requests_total{service="checkout",status="500"}` son dos series independientes, aunque compartan el mismo nombre de métrica.

Las métricas se clasifican en varios tipos, entre los cuales se destacan:

- **Counter**: contador que sólo aumenta (o se resetea a cero). Es ideal para medir la cantidad de eventos: número de requests, cantidad de errores, trabajos procesados, etc.
- **Gauge**: valor que puede subir y bajar. Se usa para representar estados: número de conexiones activas, tamaño de una cola, uso actual de memoria o CPU.
- **Histogram** y **Summary**: tipos orientados a estudiar distribuciones, muy útiles para analizar latencias. En lugar de un solo promedio, permiten ver cómo se reparten los valores en distintos rangos y calcular percentiles (p50, p95, p99).

Elegir el tipo correcto de métrica es crucial para que luego los gráficos y las consultas sean significativos. Por ejemplo, para analizar latencia suele ser preferible usar histogramas o *summaries* que permitir calcular percentiles, en lugar de guardar solo un promedio; el promedio tiende a ocultar los casos extremos que, sin embargo, impactan fuertemente en la experiencia de usuario.

Respecto de la **forma de recolección**, se habla principalmente de dos enfoques:

1. **Modelo pull**: una herramienta central (como Prometheus) consulta periódicamente a cada servicio a través de un endpoint HTTP especial (por defecto `/metrics`) y obtiene todas las métricas en un formato estándar de texto.
2. **Modelo push**: los servicios envían activamente sus métricas hacia un colector, por ejemplo, a través de un agente o un gateway.

Prometheus utiliza el modelo *pull* y, para poder ser medidas, las aplicaciones deben **instrumentarse**, es decir, incorporar bibliotecas de cliente que permitan definir métricas en el código y exponerlas en el *endpoint* de métricas. Para muchos componentes de infraestructura (sistemas operativos, bases de datos, servidores web) existen programas auxiliares llamados **exporters**, que traducen los datos de esos sistemas al formato que Prometheus entiende.

En la práctica, un flujo típico de trabajo con métricas es el siguiente:

1. Identificar qué aspectos del sistema es importante medir (latencia, tasa de errores, tráfico, uso de recursos, métricas de negocio).

2. Instrumentar la aplicación o desplegar *exporters* para recolectar esos datos.
3. Configurar el sistema de métricas para recolectarlas periódicamente.
4. Visualizarlas en *dashboards* y utilizarlas para construir alertas.

Cuando este ciclo se realiza de forma consistente, las métricas dejan de ser “números aislados” y se convierten en una fuente de información para entender el comportamiento del sistema y su evolución.

Material para profundizar

- Sitio oficial de Prometheus – página principal. <https://prometheus.io/>
- Prometheus – “Client Libraries”.
<https://prometheus.io/docs/instrumenting/clientlibs/>
- Prometheus – repositorio oficial en GitHub (descripción de arquitectura y características). <https://github.com/prometheus/prometheus>

3. Logs y su rol dentro de la telemetría

Los **logs** o registros son otra pieza esencial de la telemetría. Mientras las métricas ofrecen una visión numérica y agregada del estado del sistema, los *logs* registran eventos individuales con mucho más detalle. Un *log* típico incluye un *timestamp*, un nivel de severidad (por ejemplo, *info*, *warning*, *error*), un mensaje descriptivo y, en muchos casos, campos adicionales que aportan contexto (usuario, servicio, identificadores de operación, etc.).

En sistemas pequeños, es común que los *logs* se escriban en archivos de texto lineales. Sin embargo, a medida que las arquitecturas se vuelven distribuidas (por ejemplo, con microservicios) y el volumen de eventos crece, se vuelve recomendable utilizar **logs estructurados**, normalmente en formato JSON u otro formato parseable. Esto permite que herramientas específicas puedan filtrar, agrupar y buscar eventos de forma eficiente.

Desde la perspectiva de la observabilidad, los *logs* cumplen un rol complementario a las métricas:

- Las **métricas** se utilizan principalmente para responder preguntas del tipo “¿cuánto?” y “¿cómo cambia en el tiempo?”.
- Los **logs** se usan para responder “¿qué pasó exactamente?” y “¿en qué contexto?”.

Un flujo de trabajo muy habitual frente a un incidente es el siguiente:

1. Primero se consulta un dashboard de métricas (por ejemplo, en Grafana) para detectar qué servicio o *endpoint* está mostrando latencia alta o tasa de errores elevada.
2. Una vez localizado el problema, se filtran los *logs* correspondientes a ese servicio y rango temporal para ver los detalles de los errores, mensajes de negocio, excepciones, etc.
3. Con esa información se formula una hipótesis sobre la causa raíz y se planifica la corrección.

Es importante diseñar los *logs* con criterio. Algunas buenas prácticas son:

- Registrar eventos significativos para el negocio y para la operación (por ejemplo, errores, transacciones fallidas, cambios de estado importantes).
- Incluir identificadores que permitan correlacionar *logs* entre servicios, como un *correlation ID* o *trace ID*. De este modo, se pueden reconstruir los pasos de una operación que atraviesa múltiples componentes.
- Usar niveles de severidad de manera coherente, evitando marcar como “error” cosas que no lo son, para no generar ruido.

Los *logs* también participan de la construcción de observabilidad. Proyectos como OpenTelemetry definen los *logs*, junto con métricas y trazas, como señales de telemetría que se pueden recolectar, procesar y enviar a distintos *backends* de análisis.

En contextos de DevOps, donde los equipos se responsabilizan por el ciclo completo de vida de un servicio, diseñar y cuidar la estrategia de *logging* es fundamental. Una mala estrategia puede generar grandes volúmenes de datos poco útiles, encarecer el almacenamiento y dificultar la búsqueda de información relevante. En cambio, un enfoque ordenado permite combinar métricas y logs para llegar mucho más rápido a diagnósticos claros.

Material para profundizar

- OpenTelemetry – “Concepts” (sección sobre señales de telemetría, incluyendo logs). <https://opentelemetry.io/docs/concepts/>
- Google SRE Book – Tabla de contenidos (capítulos sobre monitoreo y SLO). <https://sre.google/sre-book/table-of-contents/>

4. Prometheus e instrumentación de aplicaciones

Prometheus es una herramienta de monitoreo basada en métricas que se convirtió en un estándar de facto en entornos cloud native. Se trata de un sistema que recolecta métricas desde distintos objetivos (*targets*), las almacena como series temporales y permite consultarlas mediante un lenguaje específico, PromQL. Prometheus se distribuye como *software* libre y forma parte de la Cloud Native Computing Foundation.

Su arquitectura básica incluye:

- Un **servidor Prometheus**, encargado de leer la configuración, realizar el *scraping* de métricas, almacenarlas y evaluar reglas.
- **Targets instrumentados**, es decir, aplicaciones y servicios que exponen sus métricas en un endpoint HTTP (`/metrics`) en un formato de texto estándar.
- **Exporters**, procesos que traducen métricas de sistemas externos (como bases de datos, sistemas operativos o servidores web) al formato Prometheus.
- **Alertmanager** (componente separado), que recibe alertas basadas en consultas de Prometheus y las envía a distintos canales (correo, chat, etc.).

El corazón del uso de Prometheus está en la **instrumentación**. Instrumentar una aplicación significa modificarla (o configurarla) para que genere métricas relevantes y las exponga en el formato correcto. Para facilitar esta tarea existen bibliotecas de cliente (*client libraries*) en varios lenguajes de programación (Go, Java, Python, Node.js, C, entre otros).

En términos generales, instrumentar implica:

1. **Incorporar la client library** de Prometheus en el proyecto.
2. **Definir métricas** (*counters, gauges, histograms*) que tengan sentido para el dominio del sistema: cantidad de *requests* por *endpoint*, latencia de operaciones clave, tamaño de colas internas, cantidad de errores, etc.
3. **Actualizar las métricas** en el código en los puntos adecuados. Por ejemplo, incrementar un *counter* al recibir una *request* o registrar el tiempo que tarda en ejecutarse una operación.
4. **Exponer un endpoint `/metrics`** que devuelva todas las métricas actuales en el formato de exposición de Prometheus.

Del lado de Prometheus, se configura el archivo `prometheus.yml` para indicar qué endpoints debe *scrapear*, cada cuánto tiempo y con qué nombre de *job* o conjunto. A partir de esa configuración, el servidor recolecta periódicamente los datos sin que la aplicación tenga que “empujarlos”.

La combinación de una instrumentación adecuada con Prometheus activo permite responder preguntas como:

- ¿Qué *endpoints* tienen mayor latencia?
- ¿En qué horarios crece la tasa de errores?
- ¿Qué servicios concentran la mayor cantidad de tráfico?
- ¿Cómo se comporta el sistema antes y después de un despliegue?

Estas respuestas no surgen de impresiones subjetivas, sino de datos concretos que pueden visualizarse y analizarse.

Material para profundizar

- Prometheus – documentación principal.
<https://prometheus.io/docs/introduction/overview/>
- Prometheus – “Client Libraries”.
<https://prometheus.io/docs/instrumenting/clientlibs/>
- Prometheus Python Client – documentación.
https://github.com/prometheus/client_python [GitHub+1](#)

5. Grafana, *dashboards* y análisis de tendencias

Grafana es una plataforma de visualización que permite construir paneles (*dashboards*) a partir de distintos orígenes de datos, entre ellos Prometheus. A través de Grafana es posible transformar métricas crudas en gráficos, tablas y paneles que resultan mucho más intuitivos a la hora de interpretar el estado de un sistema.

Un ***dashboard*** es un conjunto de paneles que comparten un mismo rango temporal y, generalmente, un conjunto de filtros comunes (por ejemplo, servicio, entorno, región). Cada panel se construye a partir de una consulta (PromQL cuando la fuente de datos es Prometheus) y una visualización específica: líneas de tiempo, barras, tablas, mapas de calor, indicadores de estado, entre otros.

Diseñar buenos *dashboards* implica pensar primero en las preguntas que se quieren responder. Algunos ejemplos:

- “¿Está cumpliendo el sistema sus objetivos de latencia y errores?”
- “¿Qué servicio es responsable de la degradación observada?”
- “¿Cómo evoluciona la carga a lo largo del día o de la semana?”
- “¿Dónde se ubica el cuello de botella: CPU, base de datos, red, disco?”

Una recomendación habitual es organizar los *dashboards* por **flujo de negocio** (por ejemplo, *login*, búsqueda, *checkout*, pago) en lugar de hacerlo solo por componentes técnicos. De este modo, cada dashboard combina métricas técnicas y métricas de negocio relacionadas con el mismo proceso, lo cual facilita la conversación con áreas no técnicas.

Otro aspecto fundamental es el uso de **rangos temporales**. Grafana permite cambiar fácilmente la ventana de tiempo: últimos 5 minutos, 1 hora, 24 horas, 7 días, etc. Un incidente en curso suele analizarse con rangos cortos, mientras que para estudiar patrones y tendencias conviene usar rangos largos. Es frecuente detectar, por ejemplo, picos de latencia o de tráfico que se repiten todos los días a la misma hora, o todos los fines de semana. Esos patrones permiten ajustar la capacidad, planificar mantenimientos o pensar estrategias de optimización específicas.

Grafana también ofrece funcionalidades adicionales como anotaciones sobre gráficos (para marcar despliegues o eventos relevantes), variables de panel que permiten cambiar fácilmente filtros sin editar consultas y bibliotecas de dashboards preconstruidos para Prometheus y otros orígenes de datos.

Más allá de las características técnicas, lo importante es entender que un dashboard no es un “adorno visual”, sino una herramienta que guía decisiones: cuándo escalar, cuándo investigar un problema, si una nueva versión mejoró o empeoró el rendimiento, etc. Cuando los equipos se acostumbran a utilizar dashboards en su trabajo cotidiano, dejan de depender de percepciones aisladas y comienzan a discutir con un lenguaje basado en datos.

Material para profundizar

- Grafana – documentación “Visualize data”. <https://grafana.com/docs/grafana/latest/visualizations/>
- Grafana – documentación de dashboards. <https://grafana.com/docs/grafana/latest/visualizations/dashboards/>
- Documentación general de Grafana (índice). <https://grafana.com/docs/>

6. Uso de los datos para identificar cuellos de botella y optimizar el rendimiento

Recolectar métricas y disponer de dashboards solo tiene sentido si esos datos se utilizan para mejorar el sistema. En este punto se aborda cómo transformar la telemetría en decisiones concretas, especialmente en lo relacionado con la identificación de cuellos de botella y la optimización del rendimiento.

Un concepto muy utilizado es el de los **“golden signals”** o señales doradas, propuesto en el ámbito del Site Reliability Engineering. Estos cuatro indicadores básicos son: latencia, tráfico, tasa de errores y saturación. Monitorear estos aspectos para cada servicio o flujo de negocio brinda una visión bastante completa de su estado.

- **Latencia:** tiempo que tarda una operación en completarse (por ejemplo, una request HTTP). No alcanza con mirar promedios; es importante observar percentiles como p95 y p99, que muestran el comportamiento de los casos más lentos.
- **Tráfico:** cantidad de trabajo que recibe el sistema (*requests* por segundo, cantidad de *jobs*, mensajes en colas).
- **Errores:** porcentaje de operaciones fallidas, normalmente medido a través de códigos de estado o métricas específicas.
- **Saturación:** grado en que los recursos están siendo utilizados (CPU, memoria, conexiones, discos, etc.).

Al analizar un incidente, se suele comenzar observando cómo se comportan estos indicadores. Por ejemplo, si el tráfico aumenta y simultáneamente la latencia y la tasa de errores suben, es probable que el sistema esté llegando a un límite de capacidad. Si, en cambio, el tráfico se mantiene estable, pero la latencia se incrementa y la saturación de la base de datos se aproxima al máximo, eso puede indicar un cuello de botella en el acceso a datos.

La **interpretación de tendencias** en el tiempo es igualmente importante. Las herramientas de visualización permiten cambiar el rango temporal para ver:

- Picos breves y aislados, que pueden asociarse a eventos puntuales.
- Tendencias crecientes y sostenidas, que generalmente requieren acciones de fondo (optimización, escalado, rediseño).
- Patrones cíclicos, como picos en horarios específicos o en ciertos días de la semana.

Además, las métricas resultan fundamentales para **evaluar cambios** en el sistema. Cada despliegue de una nueva versión, modificación de configuración o ajuste de infraestructura debería ir acompañado de un análisis “antes y después” de las métricas relevantes. Si, luego de un cambio, la latencia de un *endpoint* aumenta o la tasa de errores se dispara, los mismos dashboards que se usan en operación sirven para detectar rápidamente la regresión.

Un enfoque más avanzado consiste en definir **SLO (Service Level Objectives)**, que son objetivos numéricos de nivel de servicio, por ejemplo: “el 99% de las *requests* a este *endpoint* debe responder en menos de 300 ms”. A partir de las métricas recolectadas se calcula si el servicio está cumpliendo o no el SLO. Este enfoque ayuda a priorizar esfuerzos: no todas las degradaciones son igual de relevantes y los SLO permiten concentrarse en las que realmente impactan la experiencia del usuario o los compromisos asumidos con el negocio.

Finalmente, es importante destacar que las métricas rara vez brindan por sí solas toda la explicación de un problema. Su rol es señalar **dónde** mirar: qué servicio, qué *endpoint*, qué horario, qué patrón. A partir de esa orientación, la investigación se profundiza combinando la información de métricas con *logs* y, cuando corresponde, con trazas distribuidas. Así se construye una cadena de evidencias que lleva desde el síntoma visible hasta la causa raíz.

Cuando este ciclo de observación, análisis y acción se repite de manera sistemática, la telemetría deja de ser un conjunto de datos aislados para convertirse en un sistema de retroalimentación que guía la evolución del software y de la infraestructura.

Material para profundizar

- Google SRE Book – Capítulo “Monitoring Distributed Systems”.
<https://sre.google/sre-book/monitoring-distributed-systems/>
- Google SRE Book – Tabla de contenidos (capítulos sobre SLO y monitoreo).
<https://sre.google/sre-book/table-of-contents/> [Google SRE](#)
- Artículo “Monitoring distributed systems” (síntesis y comentarios sobre el capítulo del libro SRE). <https://danluu.com/google-sre-book/>

Bibliografía

Grafana Labs. (s. f.). *Grafana documentation*. Recuperado de <https://grafana.com/docs/> [Grafana Labs](#)

OpenTelemetry Authors. (2024). *Observability primer*. Recuperado de <https://opentelemetry.io/docs/concepts/observability-primer/> [OpenTelemetry](#)

OpenTelemetry Authors. (2024). *OpenTelemetry concepts*. Recuperado de <https://opentelemetry.io/docs/concepts/> [OpenTelemetry](#)

Prometheus Authors. (s. f.). *Prometheus: Monitoring system & time series database*. Recuperado de <https://prometheus.io/> [Prometheus](#)

Prometheus Authors. (s. f.). *Client libraries*. En *Prometheus documentation*. Recuperado de <https://prometheus.io/docs/instrumenting/clientlibs/> [Prometheus](#)

Prometheus Authors. (s. f.). *Prometheus – GitHub repository*. Recuperado de <https://github.com/prometheus/prometheus> [GitHub](#)

Google SRE Team. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. Versión en línea de capítulos seleccionados: <https://sre.google/sre-book/> [Google SRE+1](#)